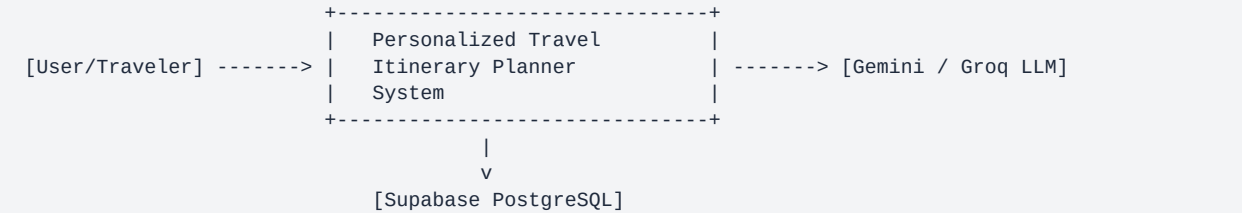
Data Flow Diagram

Date	21 October 2026
Team ID	TDS-AI-2026
Project Name	Personalized Travel Itinerary Planner
Maximum Marks	4 Marks

Project Title

Personalized Travel Itinerary Planner

Level 0: Context Diagram

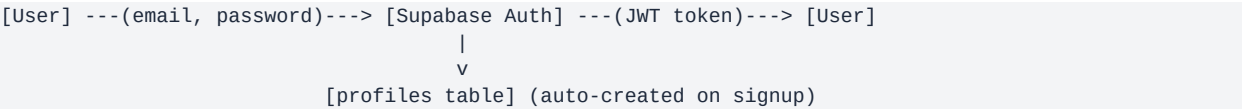


External Entities:

- User/Traveler — interacts via web browser
- Gemini / Groq LLM — generates itineraries
- Supabase PostgreSQL — stores all persistent data

Level 1: Major Data Flows

Flow 1: User Authentication



Flow 2: Trip Creation with AI Generation

Flow 3: Activity Management

```
[User] ---(edit activity)---> [Frontend] ---(PATCH /api/activities/{id})---> [Backend] ---(UPDATE)---> [DB]
[User] ---(delete activity)---> [Frontend] ---(DELETE /api/activities/{id})---> [Backend] ---(DELETE)---> [DB]
[User] ---(drag-drop reorder)---> [Frontend] ---(POST /api/trips/{id}/days/{n}/reorder)---> [Backend] ---(UPDATE positions)---> [DB]
```

Flow 4: Day Regeneration

```
[User] ---(regenerate day N)---> [Frontend] ---(POST /api/trips/{id}/days/{N}/regenerate)---> [Backend]
    |
    v
[Backend] ---(collect avoid_titles from other days)---> [Fetch existing activities]
    |
    v
[Backend] ---(single-day prompt)---> [Gemini/Groq API]
    |
    v
[Backend] ---(INSERT new activities, DELETE old activities, UPDATE day summary)---> [DB]
    |
    v
[Backend] ---(updated TripDetailOut)---> [Frontend] ---(re-rendered day)---> [User]
```

Level 2: Database Entity Relationships

```
profiles (1) -----> (N) trips
trips (1) -----> (N) itinerary_days
itinerary_days (1) -----> (N) activities
trips (1) -----> (N) trip_members
trips (1) -----> (N) generation_logs
```

Table Schemas

Table	Key Columns
profiles	id (uuid, PK), display_name, home_currency, preferences (jsonb)
trips	id (uuid, PK), owner_id (FK), title, destination, start_date, end_date, budget, currency, group_size, travel_style, interests[], notes, status
itinerary_days	id (uuid, PK), trip_id (FK), day_number, date, summary, estimated_cost
activities	id (uuid, PK), day_id (FK), position, type, title, description, location_name, start_time, end_time, estimated_cost
trip_members	trip_id + user_id (composite PK), role (editor/viewer)
generation_logs	id (uuid, PK), trip_id (FK), model, tokens_used, latency_ms, success

---

Row-Level Security (RLS) Data Access Flow

```
[API Request with JWT] ---> [FastAPI extracts user_id from token]
    |
    v
```

```
[Create per-user Supabase client with user's token]
    |
    v
[All DB queries go through RLS policies]
    |
    v
[Policies check: is_trip_owner(), has_trip_access(), can_edit_trip()]
    |
    v
[Only authorized data is returned/modified]
```